Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 1
Algoritmos Greedy

April 21, 2026

Andres Cucchiarini
113539
Manuel Manterola
113498
Facundo Cot
113443

1 Introducción

Tenemos que ayudar a Scaloni a analizar los próximos nn rivales de la selección campeona del mundo. Como técnico perfeccionista, quiere analizar videos de cada uno de los rivales. Recibió un compilado por cada rival, y necesita hacer un análisis muy detallado, lo cual le implica tomar apuntes, analizar tácticas, ver cuándo hay que hacerle un masaje a Messi, etc... Para que el análisis sea detallado, cada compilado no lo revisa únicamente él, sino también un ayudante.

El análisis del rival ii le toma sisi de tiempo a Scaloni, y luego aiai al ayudante (independientemente de cuál ayudante lo vea). Lo bueno, es que después de los grandes logros obtenidos, Scaloni cuenta con nn ayudantes (es decir, la misma cantidad que rivales), que pueden ver los videos completamente en paralelo. Siempre los ayudantes podrán ver los videos después que Scaloni haya terminado de verlo y analizarlo como corresponde (esto no lo delega). Cuando llega la hora que un ayudante lo vea, puede ser cualquiera, pero sólo uno lo verá (no hay ganancia en que dos lo vean).

El DT necesita que los rivales estén todos con sus correspondientes análisis lo antes posible, y por eso te pide que lo ayudes. Dice que confía en vos. Sabe que no lo vas a dejar tirado.

2 Respuestas

2.1 Consigna 1

Hacer un análisis del problema, y proponer un algoritmo greedy que obtenga la solución óptima al problema planteado: Dado los valores de n, los si y ai, determinar un orden en el que Scaloni debe ver los videos tal que todos los análisis estén listos lo antes posible (es decir, en el mínimo tiempo necesario).

ANÁLISIS DEL PROBLEMA

Se tiene los n rivales de la Selección Argentina de Futbol para analizar, estos seran analizados por Scaloni y sus n ayudantes, para cada uno de los rivales i tenemos que:

. Si: El tiempo que tarda Scaloni en analizarlo
. Ai: El tiempo que tarda un ayudante en completar el análisis, siempre luego que Scaloni termine con dicho rival i

Contamos con las restricciones que:

. Scaloni analiza un solo rival a la vez.
. Ayudantes de diferentes rivales i pueden trabajar en paralelo, más no sobre un mismo rival.
. Un ayudante puede comenzar con el rival, unicamente despues de que Scaloni termine.

Entonces, si un rival i ocupa la posición k en el orden elegido, tomando que Scaloni procesa todos los rivales de forma secuencial antes que el ayudante los vea, lo cual toma un tiempo acumulado. Recién a partir de ese momento el ayudante comienza su trabajo que toma un tiempo adicional Ai.

$$\left(\sum_{j=1}^{k} s_j\right) + a_i \tag{1}$$

A partir de esta expresión, se puede observar que el tiempo de finalización de cada rival depende del tiempo acumulado de Scaloni hasta ese punto, lo cual está determinado por el orden en que se procesan los rivales.

Esto implica que ubicar un rival en una posición más tardía incrementa su tiempo de finalización debido al crecimiento de la suma acumulada de los Sj.

En particular, si un rival tiene un valor de ai grande y se lo ubica en una posición avanzada en el orden, su tiempo de finalización será considerablemente mayor, ya que comenzará su procesamiento luego de una gran acumulación de tiempos previos.

Por lo tanto, se deduce que es conveniente ubicar primero aquellos rivales con mayor ai, de manera de reducir su impacto en el tiempo total final.

REGLA GREEDY

Dado que los ayudantes trabajan en paralelo, resulta conveniente que los rivales que demanden una cantidad de tiempo mas grande a los ayudantes de Scaloni comiencen lo antes posible, ya que estos pueden trabajar en paralelo si son rivales distintos y Scaloni da la orden.

Como complemento a este razonamiento teórico, se pueden considerar algunos casos particulares que permiten validar empíricamente la regla greedy propuesta. En estos casos se observa que, al ordenar los rivales de mayor a menor según Ai, se obtiene un tiempo total menor o igual al de otros órdenes posibles. Estos casos fueron verificados mediante los archivos de prueba incluidos en el repositorio.

Por ejemplo:

. En un caso donde un Ai es significativamente mayor que los demás, ubicarlo al principio evita que su ejecución retrase el tiempo final.

. En un caso donde todos los Ai son iguales, cualquier orden produce el mismo resultado, lo que indica que existen múltiples soluciones óptimas.

Por lo tanto, la regla Greedy terminaria siendo:

Ordenar los rivales de mayor a menor según Ai

2.2 Consigna 2

Demostrar por qué el algoritmo planteado obtiene siempre la solución óptima.

Puede pasar que en algunos casos haya más de una solución óptima para la resolución del problema. Nosotros, conociendo alguna de ellas, podemos demostrar a través de inversiones que nuestro algoritmo greedy siempre encuentra una óptima. Aquí la demostración:
Supongamos que tenemos una solución óptima cualquiera. Si esa solución ya está ordenada de mayor a menor según A_i, entonces coincide con la solución greedy y no hay nada más que demostrar.
Pero si no está ordenada de esa forma, entonces existe al menos una inversión. Es decir, existen dos rivales consecutivos i y j tales que $A_i < A_j$, pero aun así i aparece antes que j.
Sea T el tiempo acumulado antes de llegar a esos dos rivales. Es decir, T representa todo lo que tardó Scaloni en analizar a los rivales que aparecen antes que i y j.

Si en la solución aparece primero i y después j, entonces los tiempos de finalización de esos dos rivales son:

$$T_i = T + S_i + A_i$$

$$T_j = T + S_i + S_j + A_j$$

Como $S_j \geq 0$ y además $A_j > A_i$, se cumple que:

$$T_j = T + S_i + S_j + A_j > T_i = T + S_i + A_i$$

Entonces, en ese orden, el mayor de los dos tiempos es T_j.

Ahora intercambiamos esos dos rivales, de modo que primero quede j y después i. En ese nuevo orden, los tiempos de finalización pasan a ser:

$$T'_j = T + S_j + A_j$$

$$T'_i = T + S_j + S_i + A_i$$

Ahora vemos si este intercambio empeora o no la solución. Por un lado:

$$T'_j = T + S_j + A_j \leq T_j = T + S_i + S_j + A_j$$

porque $S_i \geq 0$. Por otro lado:

$$T'_i = T + S_j + S_i + A_i < T_j = T + S_j + S_i + A_j$$

porque $A_i < A_j$.
Entonces, tanto T'_j como T'_i son menores o iguales que T_j. Por lo tanto, al intercambiar una inversión, el tiempo total de la solución no aumenta.
Por lo tanto, si repetimos esta operación por la cantidad de elementos invertidos que haya, hasta llegar a la solución sin inversiones, nunca vamos a empeorar la solución, y si llegamos a una solución sin inversiones, llegamos a una solución ordenada por tiempo A_i mayor a menor de los rivales, que es exactamente lo que hace la solución greedy. Entonces existe una solución óptima con el mismo orden que propone el algoritmo greedy. En consecuencia, el algoritmo greedy propuesto obtiene siempre una solución óptima.

2.3 Consigna 3

Escribir el algoritmo planteado. Describir y justificar la complejidad de dicho algoritmo.

CODIGO PLANTEADO

```python
def orden_optimo(rivales):
    return rivales.sort(key=lambda rival: rival[1], reverse=True)

def tiempo_minimo(rivales):
    orden = orden_optimo(rivales)

    tiempo_scaloni = 0

    for tiempos in orden:
        Si = int(tiempos[0])
        Ai = int(tiempos[1])

        tiempo_scaloni += Si
        fin_analisis = tiempo_scaloni + Ai

    return fin_analisis
```

El algoritmo recibe una lista de rivales representados como tuplas *(Si, Ai)*. Primero los ordena de mayor a menor según Ai, siguiendo la regla greedy propuesta. Luego recorre la lista acumulando el tiempo de Scaloni y calculando, para cada rival, el instante en que finaliza su análisis completo. Durante ese recorrido se mantiene el mayor de esos tiempos, que corresponde al tiempo total final del algoritmo.

ANÁLISIS DE COMPLEJIDAD

Comenzamos recibiendo una lista de tuplas como las dadas en el ejemplo, donde cada posición corresponde a un rival i, siendo la lista de longitud n.

Esta lista es ordenada según los valores aia_iai, que representan el tiempo que tarda el ayudante en completar el análisis de cada rival. Para ello se utiliza el método sort() de Python, el cual implementa el algoritmo de Timsort, cuya complejidad es de *O(n*log(n))* (más información).

Una vez obtenida la lista ordenada de acuerdo al criterio greedy, se procede a recorrerla utilizando un ciclo for. En este recorrido se procesan los n elementos de la lista, acumulando el tiempo de Scaloni y calculando, para cada rival, su tiempo de finalización. Este paso tiene un costo lineal de *O(n)*, ya que visita cada elemento exactamente una vez.

Finalmente, el tiempo total del algoritmo está dado por la suma de ambas etapas. Sin embargo, la complejidad dominante es la del ordenamiento, ya que *O(n*log(n))* crece más rápido que O(n). Por lo tanto, la complejidad total del algoritmo es:

O(n*log(n))

2.4 Consigna 4

Analizar si (y cómo) afecta la variabilidad de los valores de *Ai* y *Si* a los tiempos del algoritmo planteado.

La variabilidad de estos dos factores puede analizarse desde dos distintas perspectivas. En primer lugar, si se considera el tiempo de ejecución del algoritmo, se observa lo siguiente:

1) Tiempo de ejecución:

. En este caso tenemos que, el tiempo de ejecución no depende de los valores de Ai y Si, ya que estos representan los tiempos asociados a cada rival i de los n que hay, pero no afectan la cantidad de operaciones que realiza el algoritmo.
. El tiempo de ejecución depende únicamente de n, es decir, de la cantidad de los rivales que se deben de procesar.
. Esto debido a que el algoritmo siempre realiza un ordenamiento de los n elementos y luego un recorrido completo de los mismos.

$$-> \textbf{Complejidad} = \textbf{O(n*log(n))}$$

La otra perspectiva desde la cual se puede analizar la influencia de Ai y de Si es la siguiente:

2) Tiempo total de análisis (resultado final del algoritmo):

. En este caso, el resultado deja de depender únicamente de n y pasa a depender directamente de los valores de Ai y de Si.
. Los valores de Ai afectar directamente el tiempo final, ya que representan el tiempo adicional que tarda el ayudante en completar el análisis.
. Los valores de Si influyen en el tiempo acumulado de Scaloni afectando el momento en que cada ayudante puede comenzar su trabajo.

La variabilidad de Ai y Si no afecta a la complejidad computacional del algoritmo, pero sí impacta en el valor final del tiempo total de análisis, modificando el resultado obtenido.

Además del análisis teórico, se realizaron pruebas empíricas utilizando distintos conjuntos de datos, variando la distribución de los valores de Ai y Si.
En particular, se consideraron casos donde:

. Los valores de Ai son similares entre Si.
. Un valor de Ai domina amplicamente a los demás.
. Los valores de Si presentan gran variabilidad

En todos los casos se observó que el tiempo de ejecución del algoritmo mantiene el mismo comportamiento asintótico, creciendo de forma consistente con ***O(n* log(n))***, independientemente de la distribución de los valores.

Sin embargo, se pudo notar que estas variaciones pueden afectar las constantes del tiempo de ejecución, es decir, el tiempo real medido, aunque sin modificar la complejidad teórica del algoritmo.

2.5 Consigna 5

Analizar si (y cómo) afecta la variabilidad de los valores de *Ai* y *Si* a la optimalidad del algoritmo planteado.

La variabilidad de los valores de Ai y de Si no afecta la optimalidad del algoritmo greedy. Esto se debe a que, como se demostró anteriormente, la corrección del algoritmo no depende de los valores específicos de estos tiempos, sino de la relación de orden entre los Ai.

Para que la solución sea óptima, lo único relevante es respetar el criterio de ordenamiento, si $Ai < Aj$, entonces conviene que el rival j sea procesado antes que el rival i. Este criterio garantiza que los trabajos con mayor tiempo asociado al ayudante comiencen lo antes posible, evitando retrasos en el tiempo total final.

Incluso en el caso en que todos los Ai sean iguales, la optimalidad del algoritmo no se ve afectada. En dicha situación, cualquier orden de los rivales produce el mismo resultado, lo que implica que existen múltiples soluciones óptimas. Aun así, el algoritmo greedy continúa siendo correcto, ya que encuentra una de estas soluciones óptimas.

2.6 Consigna 6

Realizar ejemplos de ejecución para encontrar soluciones y corroborar lo encontrado. Adicionalmente, el curso proveerá con algunos casos particulares que deben cumplirse su optimalidad también.

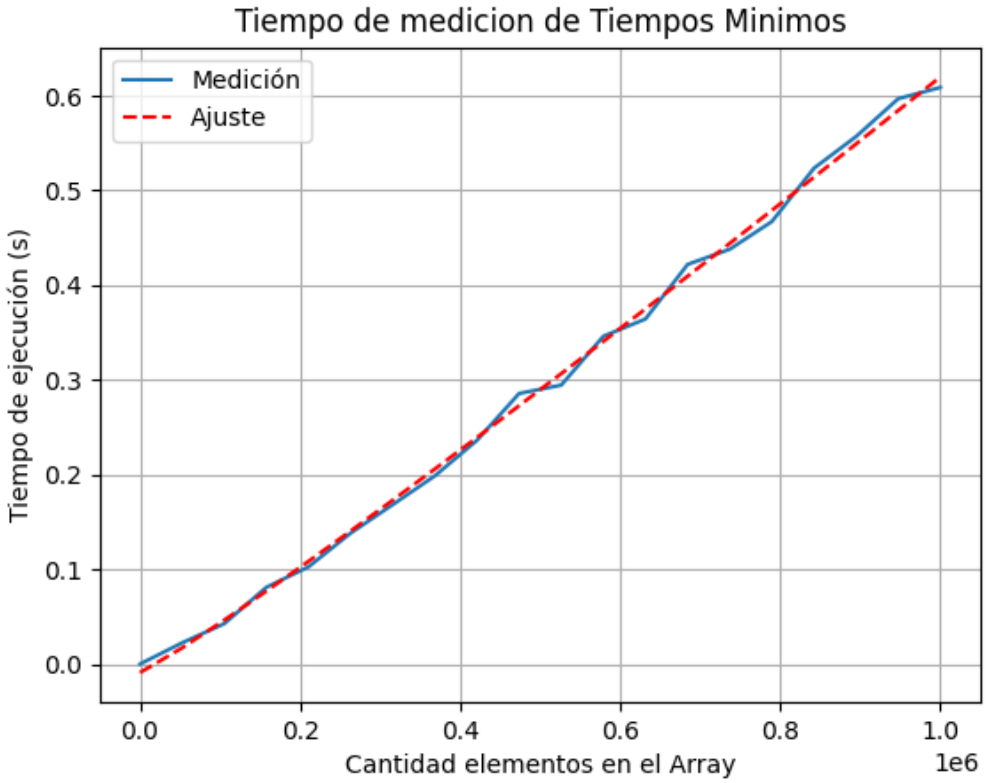
Para introducir las pruebas, lo que hicimos fue crear un programa de tests, el cual evalúa cada uno de los archivos proporcionados por la cátedra, comprobando que la salida obtenida en las distintas pruebas sea la misma que la esperada. Este programa se encuentra incluido en el repositorio y, para ejecutarlo, únicamente es necesario correr el siguiente comando desde la carpeta principal del proyecto:

− >**python3 -u tests.py**

2.7 Consigna 7

Hacer mediciones de tiempos para corroborar la complejidad teórica indicada. Esto, por supuesto, implica que deben generar sus sets de datos. Agregar los casos de prueba necesarios para dicha corroboración. Esta corroboración empírica debe realizarse confeccionando gráficos correspondientes, y utilizando la técnica de cuadrados mínimos. Para esto, proveemos una explicación detallada, en conjunto de ejemplos. Recomendamos tomar más de una medición de la misma muestra y quedarse con el promedio para reducir el ruido en la medición.

Se realizaron mediciones a partir de arreglos de diferentes tamaños, generados con 100000 (cien mil) posibilidades diferentes de horarios posibles de forma aleatoria a partir del modulo `random`.



Se puede observar que el algoritmo tiene una tendencia ***O(n*log(n))*** en función de la cantidad de elementos de entrada, como se estimo en la consigna 3.